

PHASE 3 – SYSTEM DESIGN REVIEW & DELIVERABLES

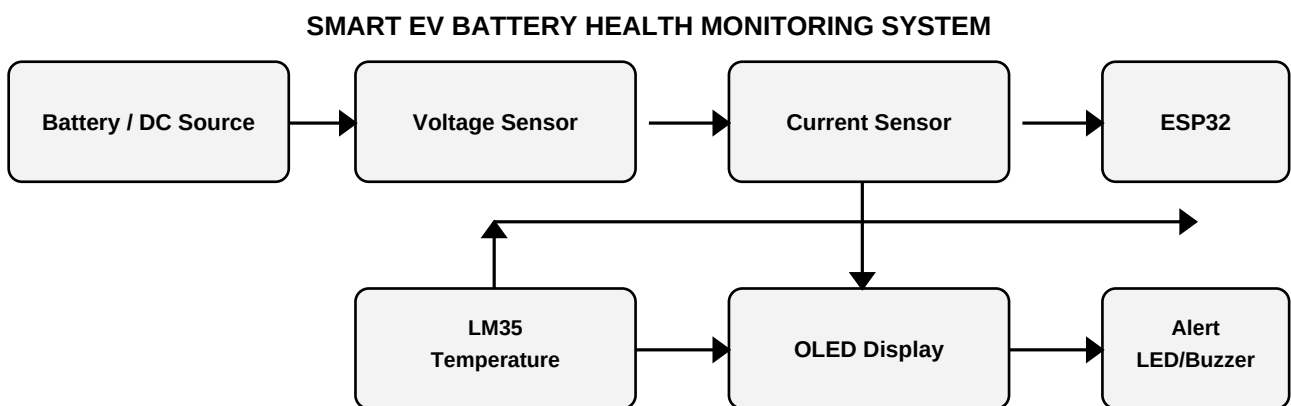
Smart Electric Vehicle Battery Health Monitoring System

Project objective: Design a prototype that continuously measures battery voltage, charging/discharging current and temperature, estimates State of Charge (SoC), displays the readings and gives an alert when an unsafe or abnormal condition is detected.

Important prototype note: This Phase 3 document is intended for a low-voltage educational prototype. Do not connect a high-voltage EV battery directly to the ESP32 or sensors. Verify wiring, sensor ratings and protection with a qualified supervisor before powering the circuit.

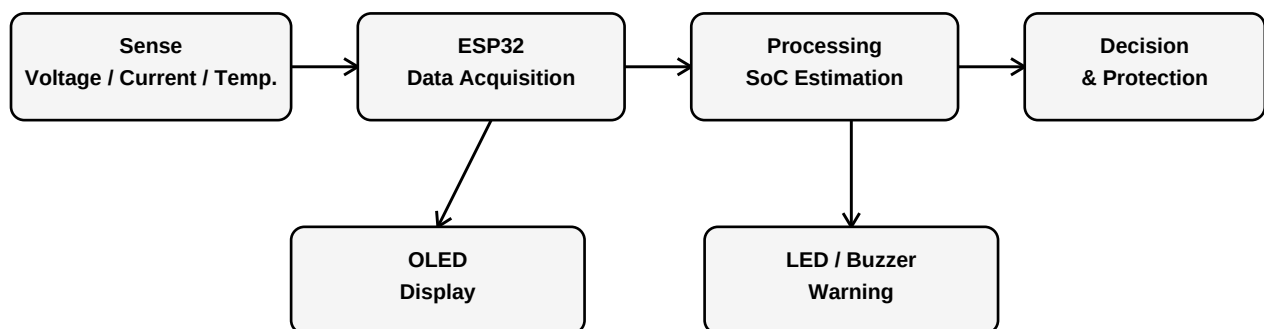
1. SYSTEM ARCHITECTURE

1.1 Overall Solution Architecture



The battery is monitored through voltage, current and temperature sensing circuits. The ESP32 collects the sensor data, estimates SoC, compares the values with predefined safe operating limits, and sends the results to the OLED display. If an abnormal condition is detected, an LED/buzzer alert is activated.

1.2 Data Flow Diagram



Data flow: Battery → Sensors → ESP32 → signal processing and SoC estimation → safety decision → OLED/alert output.

1.3 Functional Block Diagram

- **Input:** Battery voltage, battery current and battery temperature.
- **Processing:** ESP32 reads sensors, converts raw values, estimates SoC and checks thresholds.
- **Output:** OLED shows voltage, current, temperature and SoC; LED/buzzer indicates abnormal conditions.

1.4 Hardware–Software Partitioning

Hardware	Software
ESP32 WROOM	Sensor reading and calibration
Voltage sensor / divider	Voltage conversion and scaling
Current sensor (INA219/INA226 class)	Current direction and magnitude processing
LM35 temperature sensor	Temperature calculation
0.96-inch I ² C OLED	Display formatting and refresh
LED and buzzer	Threshold logic and alert control

2. HARDWARE DESIGN

2.1 Final Block Diagram

The final prototype uses an ESP32 as the central controller. The sensing stage provides voltage, current and temperature data. The OLED provides a local user interface and the LED/buzzer provides a simple warning output.

2.2 Circuit Schematic – Conceptual Connections

Module	Connection to ESP32 / system	Purpose
Voltage sensor / divider	Analog input (example: GPIO34)	Measures battery voltage after safe scaling
Current sensor	I ² C SDA/SCL or analog output, depending on module	Measures charging/discharging current
LM35	Analog input (example: GPIO35)	Measures temperature
OLED I ² C	SDA + SCL; common GND	Displays readings
LED	Digital output through suitable resistor	Visual warning
Buzzer	Digital output through suitable driver if required	Audible warning
All modules	Common reference ground where electrically appropriate	Stable signal reference

Pin numbers are example assignments, not mandatory final wiring. They must be matched to the exact ESP32 board and sensor modules used in the prototype.

2.3 Component Selection

- **ESP32 WROOM:** Main controller with ADC and I²C support; suitable for embedded monitoring and future IoT expansion.
- **Voltage sensing circuit:** Scales the battery voltage into a safe range for the ESP32 ADC.
- **INA219/INA226-class current sensor:** Provides current measurement and, depending on the module, bus-voltage information through I²C.
- **LM35:** Analog temperature sensor for monitoring battery temperature.
- **OLED 0.96-inch I²C:** Compact display for real-time readings.
- **LED + buzzer:** Local warning indicators.
- **Protection components:** Fuse/current limiting, appropriate resistors and safe connectors should be selected according to the prototype battery and sensor ratings.

2.4 Bill of Materials (BOM)

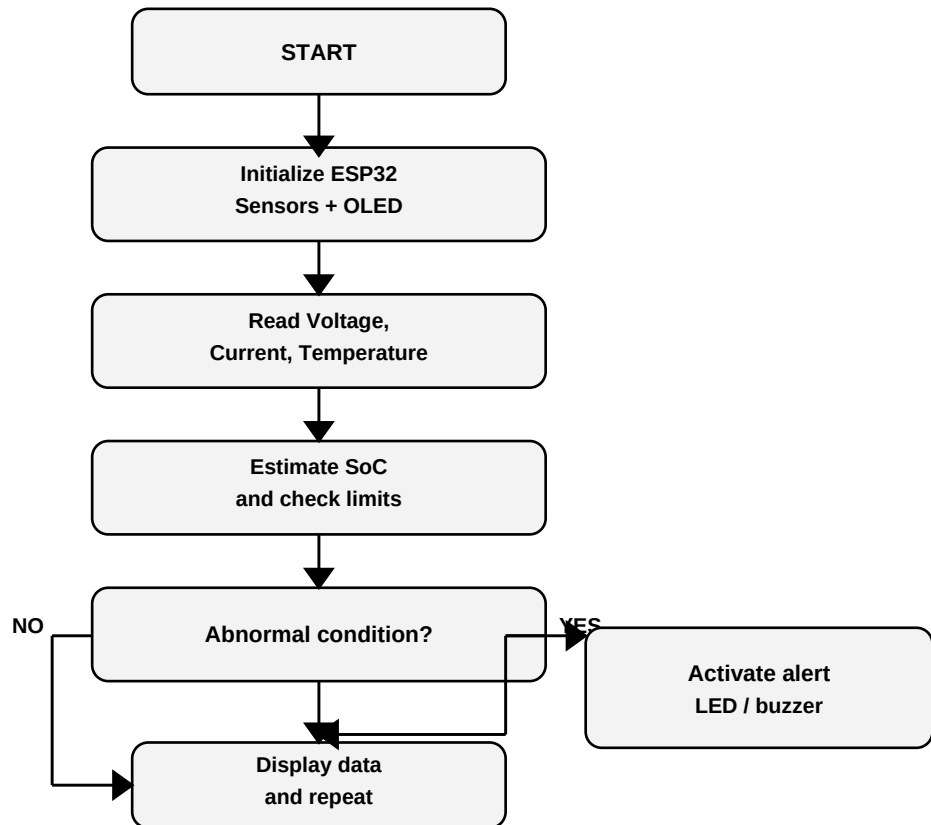
S.No.	Component	Qty.	Function
1	ESP32 WROOM development board	1	Main controller
2	Voltage sensor / resistor divider module	1	Voltage measurement
3	INA219 or INA226 current sensor	1	Current measurement
4	LM35 temperature sensor	1	Temperature measurement
5	0.96-inch I ² C OLED	1	Real-time display
6	LED	1–2	Warning indication
7	Buzzer	1	Audible warning
8	Resistors / jumper wires / breadboard	As required	Interconnection
9	Low-voltage prototype battery / DC source	1	Test source

2.5 PCB Layout (if applicable)

For the first prototype, a breadboard or perfboard implementation is sufficient. A PCB can be developed after the circuit is electrically verified. The PCB should keep sensor signal wiring short, provide clear power/ground paths, include protection components, and separate noisy/high-current paths from sensitive analog measurement lines.

3. SOFTWARE DESIGN

3.1 Flowchart



3.2 Algorithm

1. Start the system.
2. Initialize ESP32 GPIO, ADC, I²C interface, sensors and OLED.
3. Read battery voltage, current and temperature.
4. Convert sensor outputs into engineering values using calibration factors.
5. Estimate State of Charge (SoC) using the selected prototype estimation method.
6. Compare voltage, current and temperature with predefined safe limits.
7. If an abnormal condition is detected, activate the warning output.
8. Display voltage, current, temperature and SoC on the OLED.
9. Repeat the monitoring cycle continuously.

3.3 Communication Protocols

- **I²C**: Used between the ESP32 and I²C-based modules such as the OLED and INA219/INA226-class current sensor.
- **ADC**: Used by the ESP32 to read analog sensors such as LM35 and the voltage-sensing circuit.
- **UART/USB**: Can be used for serial monitoring and debugging during development.
- **Optional Wi-Fi**: The ESP32 can later transmit monitoring data to an IoT dashboard.

3.4 Database Structure (if applicable)

For the basic prototype, a database is not required. If IoT/data logging is added, a simple record can contain: timestamp, voltage, current, temperature, SoC, operating state and alert status.

Example data fields

Field	Example	Description
Timestamp	2026-09-15 16:00	Measurement time
Voltage	12.4 V	Measured battery voltage
Current	+2.5 A / -4.0 A	Charging / discharging current
Temperature	30 °C	Battery temperature
SoC	75 %	Estimated state of charge
Status	NORMAL	Current operating state
Alert	NONE	Warning condition if present

4. SIMULATION

4.1 Recommended Simulation Platform

Wokwi can be used for ESP32 logic and display testing. Proteus can be used when the required sensor/display models are available. MATLAB can be used separately to test SoC estimation or sensor-data processing algorithms.

4.2 Simulation Test Cases

Test case	Input condition	Expected output
Normal	Voltage/current/temp. within limits	OLED shows readings; no alarm
Over-voltage	Voltage above selected limit	Over-voltage warning
Under-voltage	Voltage below selected limit	Under-voltage warning
Over-current	Current above selected limit	Over-current warning
High temperature	Temperature above selected limit	Temperature warning
Charging	Positive current	Charging state indicated
Discharging	Negative current	Discharging state indicated
Idle	Current approximately zero	Idle state indicated

4.3 Initial Results – Documentation Format

- ESP32 initializes successfully and reads sensor inputs.
- OLED displays voltage, current, temperature and SoC values.
- Charging/discharging direction is identified from current measurement.
- Abnormal threshold conditions activate the alert output.
- Serial monitor can be used to compare raw sensor readings with displayed values.

Note: The above are target/initial validation results for the Phase 3 design document. Replace them with measured simulation screenshots and actual prototype values after testing.

5. DESIGN REVIEW SUMMARY

Review item	Decision / status
Architecture	ESP32-centered sensing and monitoring architecture selected
Sensors	Voltage, current and LM35 temperature sensing selected
Display	I ² C OLED selected for local real-time monitoring
Alert	LED/buzzer selected for abnormal-condition warning
Software	Continuous sensing → processing → threshold check → display
Simulation	Wokwi/Proteus recommended for initial validation
PCB	Optional; develop after breadboard verification

6. PHASE 3 DELIVERABLE CHECKLIST

- ✓ Overall Solution Architecture
- ✓ Data Flow Diagram
- ✓ Functional Block Diagram

- ✓ Hardware–Software Partitioning
- ✓ Final Block Diagram
- ✓ Circuit Schematic – conceptual connection table
- ✓ Component Selection
- ✓ Bill of Materials (BOM)
- ✓ PCB Layout guidance
- ✓ Software Flowchart
- ✓ Algorithm
- ✓ Communication Protocols
- ✓ Database Structure (optional)
- ✓ Simulation plan and test cases
- ✓ Initial-result documentation format